

Implementation Roadmap: Ebook RAG Platform (Phase 1)

Focus: Shared Collection with Metadata Filtering & Ingestion Pipeline

Following the choice of **Shared Collection with Metadata Filtering** for tenant isolation, the system will use a `tenant_id` field in every vector metadata object. All queries will be strictly scoped with a `where tenant_id == "user_123"` filter at the database level.

Step 1: Environment Orchestration (Week 1)

Before writing code, establish the infrastructure that allows for 2026-grade scaling.

- **Vector Database:** Deploy **Milvus (Standalone or Cluster)**. Create a collection named `library_vectors` with the following schema:
 - `id`: Primary Key (INT64)
 - `vector`: FloatVector (Dimension based on embedding model, e.g., 1024 for BGE-Large)
 - `tenant_id`: VarChar (Partition Key for isolation)
 - `book_id`: VarChar
 - `content_chunk`: Text
 - `metadata`: JSON (Author, Title, Page/Chapter)
- **Message Broker:** Setup **Redis**. This will act as the "waiting room" for your 10,000 ebook batches.

Step 2: The "Perfect" Ingestion Worker (Week 2)

To avoid the parsing issues found in AnythingLLM, build a Python-based worker focused on **Markdown Transformation**.

1. **Format Detection:** Use `python-magic` to verify file types.
2. **Clean Conversion:**

```
# Core Logic
if file == ".epub":
    text = extract_with_ebooklib(file)
    markdown = pandoc_convert(text, to="markdown")
elif file == ".pdf":
    markdown = pymupdf4llm.to_markdown(file)
```
3. **Semantic Chunking:** Use **LlamaIndex's SemanticSplitter**. It uses an embedding model to "look" at the text and split it only when the topic changes, ensuring your 1-2 paragraph summaries are coherent.

Step 3: Implementation of Metadata Filtering (Week 3)

Secure the search interface to ensure zero cross-talk between tenants.

- **Indexing:** When inserting vectors, the `tenant_id` must be mandatory.
- **Query Logic:** Every search request from the frontend must include the User's JWT

(JSON Web Token). The backend extracts the user_id and injects it into the Milvus query:

```
results = collection.search(  
    data=[query_vector],  
    anns_field="vector",  
    param=search_params,  
    limit=10,  
    expr=f'tenant_id == "{current_user_id}"' # Strict Isolation  
)
```

Step 4: The Summary "Agent" (Week 4)

Refine the output to meet your "1-2 paragraph" requirement.

Use **Gemini 1.5 Flash** as your reasoning engine. It has a massive context window (2M tokens) allowing you to feed it 20+ chunks of text for a highly accurate summary without losing the "story" thread.

Immediate Task	Tool/Command
1. Install local Milvus for testing	<code>docker-compose up -d</code>
2. Test PDF-to-Markdown	<code>pip install pymupdf4llm</code>
3. Initialize LlamaIndex	<code>pip install llama-index-vector-stores-milvus</code>